BÀI THỰC HÀNH CTDL
CẤU TRÚC CÂY (TREE)

Trong các buổi học trước, chúng ta đã làm việc với các cấu trúc dữ liệu tuần tự (list, stack, queue) và cấu trúc key-value (dictionary). Hôm nay, chúng ta bước sang một họ cấu trúc dữ liệu **phi tuyến** rất quan trọng: **cây (tree)**, dùng để biểu diễn dữ liệu có quan hệ phân cấp như thư mục tệp, cây gia phả, biểu thức toán học hay mục lục.

Bài thực hành này gồm các bài tập đi từ khái niệm nền tảng (nút, duyệt cây) đến cây nhị phân tìm kiếm, cây tổng quát, cây tự cân bằng AVL và cây tiền tố Trie. Toàn bộ bài tập được viết bằng **Python**, đệ quy là công cụ chủ đạo xuyên suốt.

Mục Đích và Yêu Cầu Chung

🎯 Mục đích

- **Hiểu rõ mô hình phân cấp:** Nắm vững các khái niệm gốc, nút, lá, cha-con, mức, chiều cao.
- **Thành thạo đệ quy trên cây:** Duyệt cây, tính thuộc tính cây bằng tư duy "giải quyết nút hiện tại + giao việc cho các cây con".
- **Cài đặt được BST:** Thực hiện các thao tác thêm, tìm, xóa và hiểu vì sao BST tìm kiếm nhanh.
- **Phân biệt các loại cây:** Cây nhị phân, BST, AVL, cây tổng quát, cây biểu thức, Trie và nơi ứng dụng của chúng.
- **Giải quyết bài toán thực tế:** Cây thư mục, tính toán biểu thức, gợi ý từ khóa khi tìm kiếm.

📋 Yêu cầu chung cho tất cả bài tập

- **Môi trường:** Viết chương trình dạng **Console Application** bằng Python 3.8 trở lên.
- **Code sạch:** Đặt tên biến, hàm rõ nghĩa theo chuẩn PEP 8 (snake_case). Có chú thích ở những đoạn logic quan trọng.
- **Kiểm tra nút rỗng:** Luôn xử lý trường hợp node is None trước khi truy cập node.left, node.right để tránh AttributeError.
- **Kiểm thử:** Mỗi bài phải chạy thử với ít nhất một cây mẫu, kể cả trường hợp biên (cây rỗng, cây một nút, cây lệch).
- **Học sâu với AI:** Tận dụng AI để khám phá các khái niệm như "độ phức tạp của thao tác trên cây", "cây suy biến (skewed tree)", "cây cân bằng", nhưng phải tự chạy và hiểu code trước khi nộp.

Bài tập 1: Xây dựng cây nhị phân và các phép duyệt cây - Mức độ: Dễ

Bài tập đầu tiên giúp bạn làm quen với lớp Node và ba phép duyệt cơ bản, nền tảng của mọi thuật toán trên cây.

Yêu cầu chi tiết:

- Tạo lớp Node gồm ba thuộc tính: data, left, right.
- Dựng thủ công một cây nhị phân có ít nhất 6 nút (không dùng thư viện).
- Viết ba hàm đệ quy: preorder (Gốc-Trái-Phải), inorder (Trái-Gốc-Phải), postorder (Trái-Phải-Gốc).
- In kết quả duyệt của ba phương pháp trên cùng một cây và so sánh thứ tự.

💡 Gợi ý:

- Điều kiện dừng của mọi hàm đệ quy trên cây: if node is None: return.
- Chỉ khác nhau ở vị trí dòng xử lý nút gốc so với hai lời gọi đệ quy.
- Hãy tự vẽ cây ra giấy và duyệt tay trước khi chạy chương trình để đối chiếu.

💻 Code mẫu:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None



def preorder(node, result):
    if node is None:
        return
    result.append(node.data)      # Gốc
    preorder(node.left, result)   # Trái
    preorder(node.right, result)  # Phải



def inorder(node, result):
    if node is None:
        return
    inorder(node.left, result)    # Trái
    result.append(node.data)      # Gốc
    inorder(node.right, result)   # Phải



def postorder(node, result):
    if node is None:
        return
    postorder(node.left, result)  # Trái
    postorder(node.right, result) # Phải
    result.append(node.data)      # Gốc



def main():
    #        A
    #      /   \
    #     B     C
    #    / \     \
    #   D   E     F
    root = Node("A")
    root.left = Node("B")
    root.right = Node("C")
    root.left.left = Node("D")
    root.left.right = Node("E")
    root.right.right = Node("F")

    for ten, ham in [("Preorder (NLR)", preorder),
                     ("Inorder (LNR)", inorder),
                     ("Postorder (LRN)", postorder)]:
        kq = []
        ham(root, kq)
        print(f"{ten}: {' '.join(kq)}")
```

```
if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Hỏi về lý thuyết:** Hỏi AI: *"Tại sao hàm đệ quy trên cây lại ngắn gọn như vậy? Call stack hoạt động thế nào khi duyệt inorder?"*
- **Khám phá thêm:** Hỏi AI: *"Làm thế nào để duyệt cây không dùng đệ quy mà dùng stack tự quản lý?"*

Bài tập 2: Tính các thuộc tính của cây - Mức độ: Dễ

Sau khi biết duyệt cây, ta dùng chính tư duy đệ quy đó để "hỏi" cây những câu hỏi về kích thước và hình dạng của nó.

Yêu cầu chi tiết:

- Dựng cây nhị phân chứa các số nguyên (ít nhất 6 nút).
- Viết hàm count_nodes(node): đếm tổng số nút.
- Viết hàm count_leaves(node): đếm số nút lá (nút không có con).
- Viết hàm height(node): tính chiều cao của cây.
- Viết hàm sum_tree(node) và max_value(node): tổng và giá trị lớn nhất trong cây.

💡 Gợi ý:

- Công thức chung: kết quả của một nút = kết hợp kết quả của cây con trái và cây con phải.
- Quy ước rõ ràng: cây rỗng có chiều cao -1 (hoặc 0 nếu tính theo số nút), rồi giữ nhất quán.
- Nút lá là nút có left is None and right is None.

💻 Code mẫu:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def count_nodes(node):
    if node is None:
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)


def count_leaves(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return count_leaves(node.left) + count_leaves(node.right)


def height(node):
```

```
    # Quy ước: cây rỗng có chiều cao -1, cây chỉ có gốc có chiều cao 0
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))


def sum_tree(node):
    if node is None:
        return 0
    return node.data + sum_tree(node.left) + sum_tree(node.right)


def max_value(node):
    if node is None:
        return float("-inf")
    return max(node.data, max_value(node.left), max_value(node.right))


def main():
    #          10
    #        /    \
    #       5      20
    #      / \       \
    #     3   8       30
    root = Node(10)
    root.left, root.right = Node(5), Node(20)
    root.left.left, root.left.right = Node(3), Node(8)
    root.right.right = Node(30)

    print("Số nút     :", count_nodes(root))
    print("Số nút lá  :", count_leaves(root))
    print("Chiều cao  :", height(root))
    print("Tổng giá trị:", sum_tree(root))
    print("Giá trị lớn nhất:", max_value(root))


if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Hỏi về lý thuyết:** Hỏi AI: *"Phân biệt độ sâu (depth), chiều cao (height) và mức (level) của một nút trong cây."*
- **Tư duy thuật toán:** Hỏi AI: *"Độ phức tạp thời gian và không gian của các hàm trên là bao nhiêu? Trường hợp cây lệch (skewed) thì sao?"*

Bài tập 3: Cây nhị phân tìm kiếm (BST) - Thêm và tìm kiếm - Mức độ: Dễ - Trung bình

BST là cây nhị phân có tính chất: mọi khóa ở cây con trái nhỏ hơn khóa nút gốc, mọi khóa ở cây con phải lớn hơn. Tính chất này cho phép tìm kiếm nhanh, tương tự ý tưởng tra từ điển ở tuần 6.

Yêu cầu chi tiết:

- Xây dựng lớp BST với phương thức insert(key), search(key), min_value(), max_value(), inorder().
- Không cho phép thêm khóa trùng.

- Thêm dãy số 50, 30, 70, 20, 40, 60, 80, 40 rồi in kết quả duyệt inorder (phải là dãy tăng dần).
- Cho người dùng nhập một giá trị và báo có tìm thấy trong cây hay không.

💡 Gợi ý:

- Khi thêm: nhỏ hơn thì đi sang trái, lớn hơn thì đi sang phải, gặp vị trí trống thì tạo nút mới.
- search có thể viết bằng vòng lặp while thay vì đệ quy.
- Giá trị nhỏ nhất nằm ở nút ngoài cùng bên trái, lớn nhất ở nút ngoài cùng bên phải.

💻 Code mẫu:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None



class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        self.root = self._insert(self.root, key)

    def _insert(self, node, key):
        if node is None:
            return Node(key)
        if key < node.key:
            node.left = self._insert(node.left, key)
        elif key > node.key:
            node.right = self._insert(node.right, key)
        # key == node.key: bỏ qua, không thêm trùng
        return node

    def search(self, key):
        node = self.root
        while node is not None:
            if key == node.key:
                return True
            node = node.left if key < node.key else node.right
        return False

    def min_value(self):
        node = self.root
        while node.left is not None:
            node = node.left
        return node.key

    def max_value(self):
        node = self.root
        while node.right is not None:
            node = node.right
        return node.key

    def inorder(self):
        kq = []
```

```
        def duyet(node):
            if node:
                duyet(node.left)
                kq.append(node.key)
                duyet(node.right)
        duyet(self.root)
        return kq


def main():
    bst = BST()
    for x in [50, 30, 70, 20, 40, 60, 80, 40]:
        bst.insert(x)

    print("Duyệt inorder:", bst.inorder())   # Phải là dãy tăng dần
    print("Min:", bst.min_value(), "| Max:", bst.max_value())

    x = int(input("Nhập giá trị cần tìm: "))
    print("Tìm thấy!" if bst.search(x) else "Không tìm thấy.")


if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Hỏi về lý thuyết:** Hỏi AI: *"Tại sao duyệt inorder trên BST luôn cho dãy được sắp xếp tăng dần?"*
- **So sánh:** Hỏi AI: *"So sánh chi phí tìm kiếm trong BST, trong list đã sắp xếp (tìm nhị phân) và trong dict. Khi nào nên chọn BST?"*

Bài tập 4: Xóa một nút khỏi BST - Mức độ: Trung bình

Xóa nút là thao tác khó nhất của BST vì phải giữ nguyên tính chất của cây sau khi xóa. Bài tập yêu cầu xử lý đủ ba trường hợp.

Yêu cầu chi tiết:

- Viết hàm delete(node, key) trả về gốc mới của cây con sau khi xóa.
- Xử lý ba trường hợp: (1) nút lá; (2) nút có một con; (3) nút có hai con.
- Với trường hợp hai con, thay nút bị xóa bằng **nút kế thừa inorder** (nhỏ nhất của cây con phải), rồi xóa nút kế thừa đó.
- Sau mỗi lần xóa, in duyệt inorder để kiểm tra cây vẫn hợp lệ. Thử xóa một khóa không tồn tại.

💡 Gợi ý:

- Hàm trả về nút gốc mới giúp việc "nối lại" con trỏ rất gọn: node.left = delete(node.left, key).
- Trường hợp 1 và 2 gộp lại: nếu thiếu con trái thì trả về node.right, ngược lại nếu thiếu con phải thì trả về node.left.
- Hãy vẽ sơ đồ cây trước và sau khi xóa cho từng trường hợp.

💻 Code mẫu:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None



def insert(node, key):
    if node is None:
        return Node(key)
    if key < node.key:
        node.left = insert(node.left, key)
    elif key > node.key:
        node.right = insert(node.right, key)
    return node



def min_node(node):
    while node.left is not None:
        node = node.left
    return node



def delete(node, key):
    if node is None:
        return None                         # Không tìm thấy key
    if key < node.key:
        node.left = delete(node.left, key)
    elif key > node.key:
        node.right = delete(node.right, key)
    else:
        # Trường hợp 1 và 2: nút lá hoặc chỉ có một con
        if node.left is None:
            return node.right
        if node.right is None:
            return node.left
        # Trường hợp 3: có hai con -> thay bằng nút kế thừa (nhỏ nhất bên phải)
        successor = min_node(node.right)
        node.key = successor.key
        node.right = delete(node.right, successor.key)
    return node



def inorder(node, kq):
    if node:
        inorder(node.left, kq)
        kq.append(node.key)
        inorder(node.right, kq)



def main():
    root = None
    for x in [50, 30, 70, 20, 40, 60, 80]:
        root = insert(root, x)

    for x in [20, 30, 50, 99]:              # lá, một con, hai con, không tồn tại
        root = delete(root, x)
```

```
        kq = []
        inorder(root, kq)
        print(f"Sau khi xóa {x}: {kq}")


if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Khám phá thêm:** Hỏi AI: *"Ngoài nút kế thừa inorder, còn có thể thay bằng nút tiền nhiệm (lớn nhất cây con trái) không? Hai cách khác nhau thế nào?"*
- **Kiểm thử:** Hỏi AI: *"Hãy gợi ý các test case biên cho hàm xóa BST (xóa gốc, cây một nút, cây lệch...)."*

Bài tập 5: Duyệt cây theo mức (BFS) - Mức độ: Trung bình

Khác với ba phép duyệt theo chiều sâu, duyệt theo mức đi từng tầng từ trên xuống dưới và cần một cấu trúc dữ liệu hàng đợi (queue).

Yêu cầu chi tiết:

- Cài đặt hàm level_order(root) trả về danh sách các mức, mỗi mức là một list giá trị (ví dụ [[1], [2, 3], [4, 5, 6, 7]]).
- In kết quả theo dạng "Mức 0: ..., Mức 1: ...".
- Viết thêm hàm right_side_view(root): trả về các nút nhìn thấy khi đứng bên phải cây (nút cuối cùng của mỗi mức).

💡 Gợi ý:

- Dùng collections.deque với popleft() (O(1)) thay vì list.pop(0) (O(n)).
- Để tách từng mức: tại đầu mỗi vòng lặp ngoài, lưu size = len(queue) rồi xử lý đúng size nút.
- Thứ tự đưa con vào hàng đợi: trái trước, phải sau.

💻 Code mẫu:

```
from collections import deque


class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


def level_order(root):
    """Trả về danh sách các mức, mỗi mức là một list giá trị."""
    if root is None:
        return []
    levels = []
    queue = deque([root])
    while queue:
        size = len(queue)                # Số nút của mức hiện tại
        level = []
        for _ in range(size):
```

```
            node = queue.popleft()
            level.append(node.data)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        levels.append(level)
    return levels


def right_side_view(root):
    """Các nút nhìn thấy khi đứng bên phải cây (nút cuối mỗi mức)."""
    return [level[-1] for level in level_order(root)]


def main():
    #          1
    #        /   \
    #       2     3
    #      / \   / \
    #     4   5 6   7
    #    /
    #   8
    root = Node(1)
    root.left, root.right = Node(2), Node(3)
    root.left.left, root.left.right = Node(4), Node(5)
    root.right.left, root.right.right = Node(6), Node(7)
    root.left.left.left = Node(8)

    for i, level in enumerate(level_order(root)):
        print(f"Mức {i}: {level}")
    print("Nhìn từ bên phải:", right_side_view(root))


if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Hỏi về lý thuyết:** Hỏi AI: *"Khác biệt giữa DFS và BFS trên cây? Khi nào nên dùng cái nào?"*
- **Nâng cao:** Hỏi AI: *"Làm thế nào để duyệt theo mức kiểu zigzag (mức chẵn trái sang phải, mức lẻ phải sang trái)?"*

Bài tập 6: Kiểm tra BST hợp lệ và tổ tiên chung gần nhất (LCA) - Mức độ: Trung bình - Khó

Bài tập này đòi hỏi bạn hiểu sâu định nghĩa của BST, không chỉ ở mức "con trái nhỏ hơn cha, con phải lớn hơn cha".

Yêu cầu chi tiết:

- Viết hàm is_valid_bst(root) kiểm tra một cây nhị phân có phải là BST hợp lệ hay không.
- Thử với một cây "gần giống BST" nhưng sai (ví dụ nút 12 nằm ở cây con trái của gốc 10) để hàm trả về False.
- Viết hàm lca_bst(root, p, q) tìm tổ tiên chung gần nhất của hai khóa p và q trong BST.
- Kiểm thử với ít nhất 3 cặp (p, q) khác nhau, trong đó có trường hợp một nút là tổ tiên của nút kia.

💡 Gợi ý:

- Sai lầm thường gặp: chỉ so sánh nút với con trực tiếp. Mỗi nút phải nằm trong khoảng (low, high) do **tất cả tổ tiên** quy định.
- Truyền hai tham số low và high xuống các lời gọi đệ quy, bắt đầu với -inf và +inf.
- LCA trong BST: nếu p và q cùng nhỏ hơn nút hiện tại thì đi trái, cùng lớn hơn thì đi phải, ngược lại nút hiện tại chính là LCA.

💻 Code mẫu:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None



def is_valid_bst(node, low=float("-inf"), high=float("inf")):
    """Mỗi nút phải nằm trong khoảng (low, high) do tổ tiên quy định."""
    if node is None:
        return True
    if not (low < node.key < high):
        return False
    return (is_valid_bst(node.left, low, node.key) and
            is_valid_bst(node.right, node.key, high))



def lca_bst(root, p, q):
    """Tổ tiên chung gần nhất của p và q trong BST."""
    node = root
    while node is not None:
        if p < node.key and q < node.key:
            node = node.left
        elif p > node.key and q > node.key:
            node = node.right
        else:
            return node.key              # p và q nằm hai phía (hoặc trùng nút này)
    return None



def main():
    #        20
    #       /  \
    #      10   30
    #     / \   / \
    #    5  15 25  35
    a = Node(20)
    a.left, a.right = Node(10), Node(30)
    a.left.left, a.left.right = Node(5), Node(15)
    a.right.left, a.right.right = Node(25), Node(35)
    print("Cây a hợp lệ?", is_valid_bst(a))          # True
    print("LCA(5, 15) =", lca_bst(a, 5, 15))          # 10
    print("LCA(5, 35) =", lca_bst(a, 5, 35))          # 20

    # Cây "gần giống BST" nhưng sai: 12 nằm bên trái gốc 10 dù 12 > 10
    b = Node(10)
    b.left, b.right = Node(5), Node(15)
    b.left.right = Node(12)
    print("Cây b hợp lệ?", is_valid_bst(b))           # False
```

```
if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Xử lý lỗi:** Cố tình viết is_valid_bst chỉ so sánh với con trực tiếp, rồi hỏi AI: *"Hãy tìm một cây khiến hàm này trả về sai kết quả."*
- **Mở rộng:** Hỏi AI: *"Làm thế nào tìm LCA trong cây nhị phân bất kỳ (không phải BST)?"*

Bài tập 7: Cây biểu thức (Expression Tree) - Mức độ: Khó

Cây biểu thức biểu diễn một biểu thức toán học: nút lá là toán hạng, nút trong là toán tử. Đây là ứng dụng kinh điển của cây trong trình biên dịch và máy tính bỏ túi.

Yêu cầu chi tiết:

- Nhận biểu thức **hậu tố** (postfix) từ người dùng, ví dụ 3 4 + 2 * 7 /.
- Dùng stack để dựng cây biểu thức từ biểu thức hậu tố.
- Viết hàm evaluate(node) tính giá trị biểu thức bằng đệ quy; xử lý lỗi chia cho 0.
- Viết hàm to_infix(node) (có đóng mở ngoặc đầy đủ) và to_prefix(node) để in biểu thức ở dạng trung tố và tiền tố.

💡 Gợi ý:

- Đọc từng token: toán hạng thì tạo nút và push vào stack; toán tử thì pop hai nút làm con (pop lần đầu là con **phải**), tạo nút toán tử rồi push lại.
- Kết thúc, stack còn đúng một phần tử là gốc của cây.
- Duyệt inorder cho dạng trung tố, preorder cho tiền tố, postorder cho hậu tố.

💻 Code mẫu:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


TOAN_TU = {"+", "-", "*", "/"}


def build_from_postfix(tokens):
    stack = []
    for t in tokens:
        if t in TOAN_TU:
            right = stack.pop()          # Toán hạng phải được pop trước
            left = stack.pop()
            node = Node(t)
            node.left, node.right = left, right
            stack.append(node)
        else:
            stack.append(Node(float(t)))
    return stack.pop()
```

```
def evaluate(node):
    if node.data not in TOAN_TU:
        return node.data
    a, b = evaluate(node.left), evaluate(node.right)
    if node.data == "+": return a + b
    if node.data == "-": return a - b
    if node.data == "*": return a * b
    if b == 0:
        raise ZeroDivisionError("Chia cho 0")
    return a / b


def to_infix(node):
    if node.data not in TOAN_TU:
        return f"{node.data:g}"
    return f"({to_infix(node.left)} {node.data} {to_infix(node.right)})"


def to_prefix(node):
    if node.data not in TOAN_TU:
        return f"{node.data:g}"
    return f"{node.data} {to_prefix(node.left)} {to_prefix(node.right)}"


def main():
    bieu_thuc = input("Nhập biểu thức hậu tố (cách nhau bởi dấu cách): ")
    # Ví dụ: 3 4 + 2 * 7 /   ->  ((3 + 4) * 2) / 7
    root = build_from_postfix(bieu_thuc.split())
    print("Trung tố :", to_infix(root))
    print("Tiền tố  :", to_prefix(root))
    print("Giá trị  :", evaluate(root))


if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Khám phá thêm:** Hỏi AI: *"Làm thế nào để dựng cây biểu thức từ biểu thức trung tố có ngoặc và độ ưu tiên toán tử (thuật toán Shunting-yard)?"*
- **Tối ưu:** Hỏi AI: *"Bỏ bớt các cặp ngoặc thừa khi in dạng trung tố dựa trên độ ưu tiên toán tử như thế nào?"*

Bài tập 8: Cây thư mục - Cây tổng quát (N-ary Tree) - Mức độ: Khó

Không phải cây nào cũng chỉ có hai con. Hệ thống tệp là ví dụ quen thuộc nhất của cây tổng quát: mỗi thư mục có thể chứa số lượng con tùy ý, lưu trong một danh sách.

Yêu cầu chi tiết:

- Tạo lớp TreeNode với name, size, is_folder và danh sách children.
- Dựng cây thư mục mẫu có ít nhất 3 cấp và 5 tệp.
- Viết hàm print_tree(node) in cây với thụt lề theo độ sâu.

- Viết các hàm: total_size(node) (tổng dung lượng thư mục), count_files(node) (đếm số tệp), find_path(node, name) (trả về đường dẫn đầy đủ đến một tệp, dạng root/Pictures/DaLat/anh2.jpg).

💡 Gợi ý:

- Cây tổng quát vẫn dùng đệ quy: xử lý nút hiện tại rồi lặp qua từng phần tử của children.
- Tệp là nút lá; dung lượng thư mục = tổng dung lượng các con.
- find_path cần mang theo danh sách đường đi hiện tại; gặp đúng tên thì nối bằng "/".join(path).

💻 Code mẫu:

```
class TreeNode:
    """Nút của cây tổng quát (n-ary): thư mục hoặc tệp."""
    def __init__(self, name, size=0, is_folder=False):
        self.name = name
        self.size = size                # Chỉ có ý nghĩa với tệp
        self.is_folder = is_folder
        self.children = []

    def add(self, child):
        self.children.append(child)
        return child


def print_tree(node, prefix=""):
    bieu_tuong = "[D]" if node.is_folder else "[F]"
    print(f"{prefix}{bieu_tuong} {node.name}")
    for child in node.children:
        print_tree(child, prefix + "    ")


def total_size(node):
    if not node.is_folder:
        return node.size
    return sum(total_size(c) for c in node.children)


def count_files(node):
    if not node.is_folder:
        return 1
    return sum(count_files(c) for c in node.children)


def find_path(node, name, path=None):
    path = (path or []) + [node.name]
    if node.name == name:
        return "/".join(path)
    for child in node.children:
        kq = find_path(child, name, path)
        if kq:
            return kq
    return None


def main():
```

```
    root = TreeNode("root", is_folder=True)
    docs = root.add(TreeNode("Documents", is_folder=True))
    docs.add(TreeNode("bao_cao.docx", 120))
    docs.add(TreeNode("de_cuong.pdf", 340))
    pics = root.add(TreeNode("Pictures", is_folder=True))
    trip = pics.add(TreeNode("DaLat", is_folder=True))
    trip.add(TreeNode("anh1.jpg", 2048))
    trip.add(TreeNode("anh2.jpg", 1536))
    root.add(TreeNode("readme.txt", 4))

    print_tree(root)
    print("Số tệp    :", count_files(root))
    print("Tổng dung lượng (KB):", total_size(root))
    print("Đường dẫn đến anh2.jpg:", find_path(root, "anh2.jpg"))


if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Kết nối kiến thức:** Hỏi AI: *"Cây tổng quát có thể biểu diễn lại bằng cây nhị phân theo kiểu con-trái anh-em-phải (left-child right-sibling) như thế nào?"*
- **Thực tế:** Hỏi AI: *"Dùng module os và os.walk để duyệt cây thư mục thật trên máy bằng Python."*

Bài tập 9: Cây AVL - Cây tìm kiếm tự cân bằng - Mức độ: Rất khó

Nếu thêm dãy số đã sắp xếp vào BST, cây sẽ suy biến thành danh sách liên kết và việc tìm kiếm mất O(n). Cây AVL khắc phục bằng cách tự cân bằng sau mỗi lần thêm.

Yêu cầu chi tiết:

- Mỗi nút lưu thêm thuộc tính height. Định nghĩa **hệ số cân bằng** = chiều cao cây con trái - chiều cao cây con phải.
- Viết hai phép quay: rotate_left và rotate_right.
- Viết hàm insert của AVL: sau khi thêm, cập nhật chiều cao và xử lý 4 trường hợp mất cân bằng: Left-Left, Right-Right, Left-Right, Right-Left.
- Thêm lần lượt 10, 20, 30, 40, 50, 25; sau mỗi lần thêm in gốc, chiều cao và duyệt preorder.
- So sánh chiều cao của BST thường và AVL khi thêm dãy 1..15 theo thứ tự tăng dần.

💡 Gợi ý:

- Hệ số cân bằng hợp lệ chỉ thuộc {-1, 0, 1}. Khi vượt ngưỡng mới cần quay.
- Left-Right: quay trái cây con trái, rồi quay phải nút hiện tại. Right-Left thì ngược lại.
- Hàm quay phải trả về gốc mới của cây con, nhớ cập nhật chiều cao **theo đúng thứ tự** (nút bị hạ xuống trước, nút được nâng lên sau).

💻 Code mẫu:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1                  # Chiều cao tính theo số nút
```

```
def height(n):
    return n.height if n else 0


def balance_factor(n):
    return height(n.left) - height(n.right) if n else 0


def update_height(n):
    n.height = 1 + max(height(n.left), height(n.right))


def rotate_right(y):
    x = y.left
    y.left = x.right
    x.right = y
    update_height(y)
    update_height(x)
    return x                                # x là gốc mới của cây con


def rotate_left(x):
    y = x.right
    x.right = y.left
    y.left = x
    update_height(x)
    update_height(y)
    return y


def insert(node, key):
    if node is None:
        return Node(key)
    if key < node.key:
        node.left = insert(node.left, key)
    elif key > node.key:
        node.right = insert(node.right, key)
    else:
        return node

    update_height(node)
    bf = balance_factor(node)

    if bf > 1 and key < node.left.key:       # Left-Left
        return rotate_right(node)
    if bf < -1 and key > node.right.key:     # Right-Right
        return rotate_left(node)
    if bf > 1 and key > node.left.key:       # Left-Right
        node.left = rotate_left(node.left)
        return rotate_right(node)
    if bf < -1 and key < node.right.key:     # Right-Left
        node.right = rotate_right(node.right)
        return rotate_left(node)
    return node
```

```
def preorder(node, kq):
    if node:
        kq.append(node.key)
        preorder(node.left, kq)
        preorder(node.right, kq)


def main():
    root = None
    for x in [10, 20, 30, 40, 50, 25]:
        root = insert(root, x)
        kq = []
        preorder(root, kq)
        print(f"Thêm {x:>2} -> gốc = {root.key}, cao = {root.height}, preorder = {kq}")


if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Hỏi về lý thuyết:** Hỏi AI: *"Chứng minh vì sao chiều cao cây AVL luôn là O(log n)?"*
- **Nâng cao:** Hỏi AI: *"Hãy hướng dẫn cài đặt thao tác xóa trong AVL. Nó khác thao tác thêm ở điểm nào?"*

Bài tập 10: Cây tiền tố (Trie) và gợi ý từ khóa - Mức độ: Rất khó

Trie là cây mà mỗi cạnh đại diện cho một ký tự, các từ có chung tiền tố sẽ dùng chung một nhánh. Đây là cấu trúc đứng sau tính năng gợi ý khi gõ (autocomplete) của các ô tìm kiếm.

Yêu cầu chi tiết:

- Tạo lớp TrieNode với children (dùng dict ánh xạ ký tự sang nút con) và cờ is_end.
- Cài đặt lớp Trie với insert(word), search(word) (khớp trọn từ) và starts_with(prefix).
- Cài đặt autocomplete(prefix): trả về tất cả từ bắt đầu bằng tiền tố, theo thứ tự bảng chữ cái.
- Nạp ít nhất 10 từ, cho người dùng nhập tiền tố và in danh sách gợi ý. Thử với tiền tố không tồn tại.

💡 Gợi ý:

- Ví dụ: sau khi thêm "car", search("ca") phải là False vì "ca" chỉ là tiền tố, không phải từ đã lưu (is_end của nút "a" là False).
- _find_node(prefix) đi theo các ký tự và trả về nút cuối (hoặc None); cả search, starts_with và autocomplete đều tận dụng hàm này.
- Autocomplete = tìm nút của tiền tố rồi duyệt DFS toàn bộ cây con của nút đó, duyệt sorted(children) để có thứ tự chữ cái.

💻 Code mẫu:

```
class TrieNode:
    def __init__(self):
        self.children = {}              # ký tự -> TrieNode (dùng dict như tuần 6)
        self.is_end = False
```

```
class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def _find_node(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def search(self, word):
        node = self._find_node(word)
        return node is not None and node.is_end

    def starts_with(self, prefix):
        return self._find_node(prefix) is not None

    def autocomplete(self, prefix):
        node = self._find_node(prefix)
        if node is None:
            return []
        kq = []

        def dfs(n, current):
            if n.is_end:
                kq.append(current)
            for ch in sorted(n.children):
                dfs(n.children[ch], current + ch)

        dfs(node, prefix)
        return kq


def main():
    trie = Trie()
    for w in ["car", "card", "care", "cat", "dog", "do", "dot"]:
        trie.insert(w)

    print("search('car')  :", trie.search("car"))       # True
    print("search('ca')   :", trie.search("ca"))        # False (chỉ là tiền tố)
    print("starts_with('do'):", trie.starts_with("do")) # True
    tien_to = input("Nhập tiền tố cần gợi ý: ")
    print("Gợi ý:", trie.autocomplete(tien_to))
```

```
if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **So sánh:** Hỏi AI: *"So sánh Trie và Dictionary trong bài toán tìm mọi từ theo tiền tố. Vì sao dict thường không làm tốt việc này?"*
- **Nâng cao:** Hỏi AI: *"Làm thế nào để Trie lưu thêm tần suất tìm kiếm và chỉ gợi ý 5 từ phổ biến nhất?"*